

MATH3881/5881: Statistical Machine Learning Theory

Week 5: Convolutional Neural Networks: Exercise Solutions

Sarat Moka

UNSW, Sydney

Worked solutions to the exercises in the Week 5 lecture notes. Numbering and exercise statements follow the lecture verbatim; equation numbers such as (5.3) refer to the corresponding equations in the lecture notes.

5.1 Motivation: From FNNs to CNNs

Exercise 5.1.1 (Parameter Count of a Dense Image Classifier)

Verify the ≈ 150 -million figure quoted above. A single-layer dense classifier maps a $3 \times 224 \times 224$ colour image, flattened to a vector of length $3 \cdot 224 \cdot 224 = 150,528$, to a $K = 1000$ -class probability vector via a weight matrix $W \in \mathbb{R}^{1000 \times 150,528}$ and a bias vector $b \in \mathbb{R}^{1000}$.

- Compute the total number of learnable parameters in (W, b) .
- At 4 bytes per parameter (32-bit float), how much memory (in MB) does this single layer require?

Solution.

- The weight matrix has $1000 \times 150,528 = 150,528,000$ entries, and the bias vector has 1,000 entries. Total learnable parameters:

$$1000 \times 150,528 + 1,000 = 150,528,000 + 1,000 = 150,529,000.$$

This is $\approx 1.5 \times 10^8$, confirming the ≈ 150 -million figure quoted in the lecture.

- At 4 bytes per parameter:

$$150,529,000 \times 4 = 602,116,000 \text{ bytes} \approx 602 \text{ MB},$$

using $1 \text{ MB} = 10^6$ bytes. A *single* classifier layer therefore requires about 600 MB just to store the parameters, before accounting for activations, gradients, or the optimiser state, which typically doubles or triples this footprint during training. This is a striking illustration of why FNNs applied naively to images do not scale.

5.2 The Convolution Operation

Exercise 5.2.1 (Computing a 2D Convolution)

Let $W = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ be a 2×2 kernel and $x = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ a 3×3 input. Use (5.3) to compute all four entries $z_{1,1}, z_{1,2}, z_{2,1}, z_{2,2}$ of $z = W \star x$, and verify that the output dimensions match (5.4).

Solution. *Step 1: output dimensions.* By (5.4), the output has size $(M_h - K_h + 1) \times (M_v - K_v + 1) = (3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$.

Step 2: instantiate (5.3) for $K_h = K_v = 2$. The formula

$$z_{i',j'} = \sum_{i=0}^{K_h-1} \sum_{j=0}^{K_v-1} w_{K_h-i, K_v-j} x_{i'+i, j'+j}$$

expands to

$$z_{i',j'} = w_{2,2} x_{i',j'} + w_{2,1} x_{i',j'+1} + w_{1,2} x_{i'+1,j'} + w_{1,1} x_{i'+1,j'+1}.$$

Reading off the kernel entries $w_{1,1} = 1$, $w_{1,2} = 0$, $w_{2,1} = 0$, $w_{2,2} = 1$:

$$z_{i',j'} = x_{i',j'} + x_{i'+1,j'+1}.$$

Step 3: evaluate at the four output positions.

$$\begin{aligned} z_{1,1} &= x_{1,1} + x_{2,2} = 1 + 5 = 6, \\ z_{1,2} &= x_{1,2} + x_{2,3} = 2 + 6 = 8, \\ z_{2,1} &= x_{2,1} + x_{3,2} = 4 + 8 = 12, \\ z_{2,2} &= x_{2,2} + x_{3,3} = 5 + 9 = 14. \end{aligned}$$

So

$$z = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}.$$

Interpretation. The identity kernel $W = I$ sums the diagonal of each 2×2 patch of the input: each output entry is $x_{i',j'} + x_{i'+1,j'+1}$, the trace of the corresponding patch. A different kernel would compute a different linear combination of the same four positions.

5.3 From Fully Connected Layers to Convolutional Layers

Exercise 5.3.1 (FC vs Conv Parameter Count)

Consider a single layer mapping a 4×4 grayscale image to a 3×3 grayscale output.

- As a fully connected layer (5.5) (one weight per output-input pixel pair, plus one bias per output neuron), how many learnable parameters does it have?
- As a convolutional layer with a 2×2 kernel and one shared bias, how many learnable parameters does it have? By what factor has the parameter count dropped?

Solution.

- (a) The fully connected layer has one independent weight per (output pixel, input pixel) pair, plus one bias per output neuron:

$$\underbrace{(3 \cdot 3) \times (4 \cdot 4)}_{\text{weights}} + \underbrace{3 \cdot 3}_{\text{biases}} = 9 \times 16 + 9 = 144 + 9 = 153.$$

- (b) The convolutional layer has a single 2×2 kernel (four independent weights) and one shared bias:

$$2 \times 2 + 1 = 5.$$

Reduction factor: $153/5 = 30.6$, roughly a **30-fold** reduction. Translation invariance and locality together replace an unrestricted 9×16 weight matrix by a single reusable 2×2 kernel.

5.4 Building a Convolutional Layer

Exercise 5.4.1 (Padding for Size Preservation)

For an input of size 28×28 and a 5×5 kernel (stride 1), find the padding (p_h, p_v) that makes the output also 28×28 . Verify using (5.9).

Solution. *Step 1: set up the equation.* By (5.9) (with $s_h = s_v = 1$), the padded output size is

$$(M_h - K_h + p_h + 1) \times (M_v - K_v + p_v + 1).$$

For $M_h = M_v = 28$ and $K_h = K_v = 5$, requiring the output to equal 28×28 gives

$$28 - 5 + p_h + 1 = 28 \quad \text{and} \quad 28 - 5 + p_v + 1 = 28.$$

Step 2: solve. Both equations give $p_h = p_v = 4$.

Verification. $(28 - 5 + 4 + 1) \times (28 - 5 + 4 + 1) = 28 \times 28$, as required. Consistent with the general rule $(p_h, p_v) = (K_h - 1, K_v - 1)$ for stride-1 dimension preservation.

Convention note (important when you translate to code). In the lecture, p_h denotes the *total* padding across both sides of the input, so $p_h = 4$ here means two rows of zeros above and two below. Deep-learning frameworks such as PyTorch and TensorFlow use the opposite convention: their **padding** argument is the number of zeros *per side*. If you reproduce this exercise in PyTorch, use `Conv2d(..., padding=2)`, not `padding=4`. The same translation ($p_{\text{lecture}} = 2 p_{\text{PyTorch}}$) applies throughout the remaining exercises in this section.

Exercise 5.4.2 (Output Size with Stride)

For an input of size 32×32 , a 3×3 kernel, padding $(p_h, p_v) = (2, 2)$, and stride $(s_h, s_v) = (2, 2)$, compute the output spatial size using (5.10).

Solution. *Step 1: apply (5.10).* The strided output size is

$$M_h^{[1]} = 1 + \left\lfloor \frac{M_h - K_h + p_h}{s_h} \right\rfloor.$$

Substituting $M_h = 32$, $K_h = 3$, $p_h = 2$, $s_h = 2$:

$$M_h^{[1]} = 1 + \left\lfloor \frac{32 - 3 + 2}{2} \right\rfloor = 1 + \left\lfloor \frac{31}{2} \right\rfloor = 1 + 15 = 16.$$

Step 2: By symmetry $M_v^{[1]} = 16$, so the output is **16×16** .

Interpretation. A stride of 2 approximately halves each spatial dimension; here $32 \rightarrow 16$, i.e., exactly halved, because the padding just compensates for the shrinkage that a stride-1 pass would otherwise produce.

Exercise 5.4.3 (Dilated vs Standard Kernel)

A 3×3 kernel with dilation $(d_h, d_v) = (2, 2)$ covers what spatial region of the input? Compare the parameter count of this dilated 3×3 kernel against a non-dilated kernel that covers the same region.

Solution. *Step 1: effective coverage.* A dilated kernel of size $K_h \times K_v$ with dilation (d_h, d_v) covers a spatial region of size

$$(d_h(K_h - 1) + 1) \times (d_v(K_v - 1) + 1).$$

For $K_h = K_v = 3$ and $d_h = d_v = 2$: $(2 \cdot 2 + 1) \times (2 \cdot 2 + 1) = \mathbf{5 \times 5}$.

Step 2: parameter counts.

- Dilated 3×3 kernel: $K_h \cdot K_v = 9$ trainable weights.
- Non-dilated 5×5 kernel covering the same 5×5 region: 25 trainable weights.

Step 3: comparison. The dilated design achieves the same receptive field with $9/25 = 36\%$ of the parameters, a factor of about $2.8\times$ fewer. The trade-off is sparsity: only 9 of the 25 positions inside the 5×5 receptive field contribute to each output pixel, so fine-grained detail at the intermediate positions is ignored. Dilated convolutions are used when large receptive fields are needed cheaply, for example in semantic segmentation networks, but a non-dilated kernel is preferable when every pixel of the receptive field must contribute.

5.5 CNN Architectures: Pooling, VGG19, and ResNets

Exercise 5.5.1 (VGG19 Analysis and the ResNet Gradient)

- (a) Using (5.14), verify the parameter count for layers 1 and 2 of VGG19 in Table 5.1: layer 1 has $M_c^{[0]} = 3$, $M_c^{[1]} = 64$, 3×3 kernels; layer 2 has $M_c^{[0]} = 64$, $M_c^{[1]} = 64$. What fraction of VGG19's ≈ 143.5 million total parameters do these two layers account for?
- (b) Dense layer 22 maps a vector of length 25,088 to 4,096 outputs. Using the dense-layer formula $N^{[\ell-1]} \cdot N^{[\ell]} + N^{[\ell]}$, compute its parameter count and determine what percentage of VGG19's total parameter budget it consumes.

Solution. *Erratum in the lecture notes.* In Table 5.1 of the released notes, the parameter count for layers 12–15 should be $1,180,160 + 3 \times 2,359,808 = 8,259,584$, not 7,079,936, and the correct total is therefore 143,667,240 (≈ 143.7 million) rather than the printed 143,487,592. The fractions below use the corrected total; the conclusions are unaffected.

- (a) Formula (5.14) gives per-layer parameter count $M_c^{[1]}(M_c^{[0]} K_h K_v + 1)$ (kernel weights across all input-output channel pairs, plus one bias per output channel).

Layer 1 ($M_c^{[0]} = 3$, $M_c^{[1]} = 64$):

$$64 \cdot (3 \cdot 3 \cdot 3 + 1) = 64 \cdot 28 = 1,792.$$

Layer 2 ($M_c^{[0]} = 64$, $M_c^{[1]} = 64$):

$$64 \cdot (64 \cdot 3 \cdot 3 + 1) = 64 \cdot 577 = 36,928.$$

Total for layers 1 and 2: $1,792 + 36,928 = 38,720$, matching Table 5.1.

Fraction of VGG19's total: $38,720/143,667,240 \approx 0.027\%$, less than 0.03% of all parameters.

- (b) Dense layer 22 has

$$N^{[\ell-1]} \cdot N^{[\ell]} + N^{[\ell]} = 25,088 \times 4,096 + 4,096 = 102,760,448 + 4,096 = 102,764,544 \text{ parameters.}$$

As a percentage: $102,764,544/143,667,240 \approx 71.5\%$ of the VGG19 total, i.e., this *single* dense layer holds nearly three-quarters of the entire network's parameters.